4
Sorting

Sorting is the most fundamental algorithmic problem in computer science and a rich source of programming problems for two distinct reasons. First, sorting is a useful operation which efficiently solves many tasks that every programmer encounters. As soon as you recognize your job is a special case of sorting, proper use of library routines make short work of the problem.

Second, literally dozens of different sorting algorithms have been developed, each of which rests on a particular clever idea or observation. Most algorithm design paradigms lead to interesting sorting algorithms, including divide-and-conquer, randomization, incremental insertion, and advanced data structures. Many interesting programming/mathematical problems follow from properties of these algorithms.

In this chapter, we will review the primary applications of sorting, as well as the theory behind the most important algorithms. Finally, we will describe the sorting library routines provided by all modern programming languages, and show how to use them on a non-trivial problem.

4.1 Sorting Applications

The key to understanding sorting is seeing how it can be used to solve many important programming tasks:

- *Uniqueness Testing* — How can we test if the elements of a given collection of items S are all distinct? Sort them into either increasing or decreasing order so that any repeated items will fall next to each other. One pass through the elements testing if $S[i] = S[i+1]$ for any $1 \leq i < n$ then finishes the job.

- *Deleting Duplicates* — How can we remove all but one copy of any repeated elements in S? Sort and sweep again does the job. Note that the sweeping is best done by maintaining two indices — *back*, pointing to the last element in the cleaned-out prefix array, and i, pointing to the next element to be considered. If $S[back] <> S[i]$, increment *back* and copy $S[i]$ to $S[back]$.
- *Prioritizing Events* — Suppose we are given a set of jobs to do, each with its own deadline. Sorting the items according to the deadline date (or some related criteria) puts the jobs in the right order to process them. Priority queue data structures are useful for maintaining calendars or schedules when there are insertions and deletions, but sorting does the job if the set of events does not change during execution.
- *Median/Selection* — Suppose we want to find the kth largest item in set S. After sorting the items in increasing order, this fellow sits in location $S[k]$. This approach can be used to find (in a slightly inefficient manner) the smallest, largest, and median elements as special cases.
- *Frequency Counting* — Which is the most frequently occurring element in S, i.e., the mode? After sorting, a linear sweep lets us count the number of times each element occurs.
- *Reconstructing the Original Order* — How can we restore the original arrangement of a set of items after we permute them for some application? Add an extra field to the data record for the item, such that the ith record sets this field to i. Carry this field along whenever you move the record, and later sort on it when you want the initial order back.
- *Set Intersection/Union* — How can we intersect or union the elements of two containers? If both of them have been sorted, we can merge them by repeatedly taking the smaller of the two head elements, placing them into the new set if desired, and then deleting the head from the appropriate list.
- *Finding a Target Pair* — How can we test whether there are two integers $x, y \in S$ such that $x + y = z$ for some target z? Instead of testing all possible pairs, sort the numbers in increasing order and sweep. As $S[i]$ increases with i, its possible partner j such that $S[j] = z - S[i]$ decreases. Thus decreasing j appropriately as i increases gives a nice solution.
- *Efficient Searching* — How can we efficiently test whether element s is in set S? Sure, ordering a set so as to permit efficient binary search queries is perhaps the most common application of sorting. Just don't forget all the others!

4.2 Sorting Algorithms

You have quite possibly seen a dozen or more different algorithms for sorting data. Do you remember bubblesort, insertion sort, selection sort, heapsort, mergesort, quicksort,

radix sort, distribution/bin sort, Shell sort, in-order tree traversal, and sorting networks? Most likely your eyes started to glaze by the time you made it halfway through the list. Who needs to know so many ways to do the same thing, especially when there already exists a sorting library function included with your favorite programming language?

The real reason to study sorting algorithms is that the *ideas* behind them reappear as the ideas behind algorithms for many other problems. Understand that heapsort is *really* about data structures, that quicksort is *really* about randomization, and that mergesort is *really* about divide-and-conquer, and you have a wide range of algorithmic tools to work with.

We review a few particularly instructive algorithms below. Be sure to note what useful properties (such as minimizing data movement) come with each algorithm.

- *Selection Sort* — This algorithm splits the input array into sorted and unsorted parts, and with each iteration finds the smallest element remaining in the unsorted region and moves it to the end of the sorted region:

```
selection_sort(int s[], int n)
{
        int i,j;                /* counters */
        int min;                /* index of minimum */

        for (i=0; i<n; i++) {
                min=i;
                for (j=i+1; j<n; j++)
                        if (s[j] < s[min]) min=j;
                swap(&s[i],&s[min]);
        }
}
```

  Selection sort makes a lot of comparisons, but is quite efficient if all we count are the *number* of data moves. Only $n-1$ swaps are performed by the algorithm, which is necessary in the worst case; think about sorting a reversed permutation. It also provides an example of the power of advanced data structures. Using an efficient priority queue to maintain the unsorted portion of the array suddenly turns $O(n^2)$ selection sort into $O(n \lg n)$ heapsort!

- *Insertion Sort* — This algorithm also maintains sorted and unsorted regions of the array. In each iteration, the next unsorted element moves up to its appropriate position in the sorted region:

```
insertion_sort(int s[], int n)
{
        int i,j;                /* counters */

        for (i=1; i<n; i++) {
                j=i;
                while ((j>0) && (s[j] < s[j-1])) {
```

```
                        swap(&s[j],&s[j-1]);
                        j = j-1;
                }
        }
}
```

Insertion sort is particularly significant as the algorithm which minimizes the *amount* of data movement. An *inversion* in a permutation p is a pair of elements which are out of order, i.e., an i, j such that $i < j$ yet $p[i] > p[j]$. Each swap in insertion sort erases exactly one inversion, and no element is otherwise moved, so the number of swaps equals the number of inversions. Since an almost-sorted permutation has few inversions, insertion sort can be very effective on such data.

- *Quicksort* — This algorithm reduces the job of sorting one big array into the job of sorting two smaller arrays by performing a *partition* step. The partition separates the array into those elements that are less than the pivot/divider element, and those which are strictly greater than this pivot/divider element. Because no element need ever move out of its region after the partition, each subarray can be sorted independently. To facilitate sorting subarrays, the arguments to `quicksort` include the indices of the first (`l`) and last (`h`) elements in the subarray.

```
quicksort(int s[], int l, int h)
{
        int p;                  /* index of partition */

        if ((h-l)>0) {
                p = partition(s,l,h);
                quicksort(s,l,p-1);
                quicksort(s,p+1,h);
        }
}

int partition(int s[], int l, int h)
{
        int i;                  /* counter */
        int p;                  /* pivot element index */
        int firsthigh;          /* divider position for pivot */

        p = h;
        firsthigh = l;
        for (i=l; i<h; i++)
                if (s[i] < s[p]) {
                        swap(&s[i],&s[firsthigh]);
                        firsthigh ++;
                }
        swap(&s[p],&s[firsthigh]);
```

```
        return(firsthigh);
}
```

Quicksort is interesting for several reasons. When implemented properly, it is the fastest in-memory sorting algorithm. It is a beautiful example of the power of recursion. The `partition` algorithm is useful for many tasks in its own right. For example, how might you separate an array containing just 0's and 1's into one run of each symbol?

4.3 Program Design Example: Rating the Field

Pretty Polly has no shortage of gentlemen suitors who come a' courting. Indeed, her biggest problem is keeping track of who the best ones are. She is smart enough to realize that a program which ranks the men from most to least desirable would simplify her life. She is also persuasive enough to have talked you into writing the program.

Polly really likes to dance, and has determined the optimal partner height is 180 centimeters tall. Her first criteria is finding someone who is as close as possible to this height; whether they are a little taller or shorter doesn't matter. Among all candidates of the same height, she wants someone as close as possible to 75 kilograms without going over. If all equal-height candidates are over this limit, she will take the lightest of the bunch. If two or more people are identical by all these characteristics, sort them by last name, then by first name if it is necessary to break the tie.

Polly is only interested in seeing the candidates ranked by name, so the input file:

```
George Bush             195     110
Harry Truman            180     75
Bill Clinton            180     75
John Kennedy            180     65
Ronald Reagan           165     110
Richard Nixon           170     70
Jimmy Carter            180     77
```

yields the following output:

```
Clinton, Bill
Truman, Harry
Kennedy, John
Carter, Jimmy
Nixon, Richard
Bush, George
Reagan, Ronald
```

Solution starts below

The key to this problem is sorting under a fairly complex criteria defined over multiple fields. There are at least two different ways we can do this. The first makes multiple sorting passes, sorting first on the *least* important key, then the next least important key, and so on until we finally sort on the major key.

Why do we sort in this order? The minor keys are only used to break ties among the major key ordering. Provided our sorting algorithm is *stable*, meaning it preserves the relative order of equal keys, our work on the minor keys remains intact if it is relevant to the final answer.

Not all sorting algorithms are stable: indeed most fast ones are not! The `insertion_sort` and `selection_sort` functions from Section 4.2 are stable, while `quicksort` is not stable. Before you assume the stability of any sorting function, check the documentation carefully.

The other approach, which is what we opted for, rolls all the keys up into one complex comparison function. Doing it this way made it easiest to take advantage of the C library sorting routine, described in the next section.

4.4 Sorting Library Functions

Whenever possible, take advantage of the built-in sorting/searching libraries in your favorite programming language:

Sorting and Searching in C

The `stdlib.h` contains library functions for sorting and searching. For sorting, there is the function `qsort`:

```
#include <stdlib.h>

void qsort(void *base, size_t nel, size_t width,
          int (*compare) (const void *, const void *));
```

The key to using `qsort` is realizing what its arguments do. It sorts the first `nel` elements of an array (pointed to by `base`), where each element is `width`-bytes long. Thus we can sort arrays of 1-byte characters, 4-byte integers, or 100-byte records, all by changing the value of `width`.

The ultimate desired order is determined by the function `compare`. It takes as arguments pointers to two `width`-byte elements, and returns a negative number if the first belongs before the second in sorted order, a positive number if the second belongs before the first, or zero if they are the same.

Here is a comparison function for sorting integers in increasing order:

```
int intcompare(int *i, int *j)
{
    if (*i > *j) return (1);
    if (*i < *j) return (-1);
```

```
    return (0);
}
```

This comparison function can be used to sort an array `a`, of which the first `cnt` elements are occupied, as follows:

```
qsort((char *) a, cnt, sizeof(int), intcompare);
```

A more sophisticated example of `qsort` in action appears in Section 4.5. The name `qsort` suggests that quicksort is the algorithm implemented in this library function, although this is usually irrelevant to the user.

Note that `qsort` destroys the contents of the original array, so if you need to restore the original order, make a copy or add an extra field to the record as described in Section 4.1.

Binary search is an amazingly tricky algorithm to implement correctly under pressure. The best solution is not to try, since the `stdlib.h` library contains an implementation called `bsearch()`. Except for the search key, the arguments are the same as for `qsort`. To search in the previously sorted array, try

```
bsearch(key, (char *) a, cnt, sizeof(int), intcompare);
```

Sorting and Searching in C++

The C++ Standard Template Library (STL), discussed in Section 2.2.1, includes methods for sorting, searching, and more. Serious C++ users should get familiar with STL.

To sort with STL, we can either use the default comparison (e.g., $\leq$) function defined for the class, or override it with a special-purpose comparison function `op`:

```
void sort(RandomAccessIterator bg, RandomAccessIterator end)
void sort(RandomAccessIterator bg, RandomAccessIterator end,
          BinaryPredicate op)
```

STL also provides a stable sorting routine, where keys of equal value are guaranteed to remain in the same relative order. This can be useful if we are sorting by multiple criteria:

```
void stable_sort(RandomAccessIterator bg, RandomAccessIterator end)
void stable_sort(RandomAccessIterator bg, RandomAccessIterator end,
          BinaryPredicate op)
```

Other STL functions implement some of the applications of sorting described in Section 4.1, including,

- `nth_element` – Return the nth largest item in the container.
- `set_union`, `set_intersection`, `set_difference` – Construct the union, intersection, and set difference of two containers.
- `unique` – Remove all consecutive duplicates.

Sorting and Searching in Java

The `java.util.Arrays` class contains various methods for sorting and searching. In particular,

```
static void sort(Object[] a)
static void sort(Object[] a, Comparator c)
```

sorts the specified array of objects into ascending order using either the natural ordering of its elements or a specific comparator c. Stable sorts are also available.

Methods for searching a sorted array for a specified object using either the natural comparison function or a new comparator c are also provided:

```
binarySearch(Object[] a, Object key)
binarySearch(Object[] a, Object key, Comparator c)
```

4.5 Rating the Field

Our solution to Polly's dating difficulties revolved around making the multi-criteria sorting step as simple as possible. First, we had to set up the basic data structures:

```
#include <stdio.h>
#include <string.h>

#define NAMELENGTH      30              /* maximum length of name */
#define NSUITORS        100             /* maximum number of suitors */

#define BESTHEIGHT      180             /* best height in centimeters */
#define BESTWEIGHT      75              /* best weight in kilograms */

typedef struct {
        char first[NAMELENGTH];         /* suitor's first name */
        char last[NAMELENGTH];          /* suitor's last name */
        int height;                     /* suitor's height */
        int weight;                     /* suitor's weight */
} suitor;

suitor suitors[NSUITORS];               /* database of suitors */
int nsuitors;                           /* number of suitors */
```

Then we had to read the input. Note that we did not store each fellow's actual height and weight! Polly's rating criteria for heights and weights were quite fussy, revolving around how these quantities compare to a reference height/weight instead of a usual linear order (i.e., increasing or decreasing). Instead, we altered each height and weight appropriately so the quantities were linearly ordered by desirability:

```
read_suitors()
{
        char first[NAMELENGTH], last[NAMELENGTH];
        int height, weight;

        nsuitors = 0;

        while (scanf("%s %s %d %d\n",suitors[nsuitors].first,
               suitors[nsuitors].last, &height, &weight) != EOF) {
                suitors[nsuitors].height = abs(height - BESTHEIGHT);
                if (weight > BESTWEIGHT)
                        suitors[nsuitors].weight = weight - BESTWEIGHT;
                else
                        suitors[nsuitors].weight = - weight;

                nsuitors ++;
        }
}
```

Finally, observe that we used `scanf` to read the first and last names as tokens, instead of character by character.

The critical comparison routine takes a pair of suitors a and b, and decides whether a is better, b is better, or they are of equal rank. To satisfy the demands of `qsort`, we must assign -1, 1, and 0 in these three cases, respectively. The following comparison function does the job:

```
int suitor_compare(suitor *a, suitor *b)
{
        int result;                     /* result of comparison */

        if (a->height < b->height) return(-1);
        if (a->height > b->height) return(1);

        if (a->weight < b->weight) return(-1);
        if (a->weight > b->weight) return(1);

        if ((result=strcmp(a->last,b->last)) != 0) return result;

        return(strcmp(a->first,b->first));
}
```

With the comparison function and input routines in place, all that remains is a driver program which actually calls `qsort` and produces the output:

```
main()
{
        int i;                          /* counter */
```

```
        int suitor_compare();

        read_suitors();

        qsort(suitors, nsuitors, sizeof(suitor), suitor_compare);

        for (i=0; i<nsuitors; i++)
                printf("%s, %s\n",suitors[i].last, suitors[i].first);
}
```

4.6 Problems

4.6.1 Vito's Family

PC/UVa IDs: 110401/10041, **Popularity:** A, **Success rate:** high **Level:** 1

The famous gangster Vito Deadstone is moving to New York. He has a very big family there, all of them living on Lamafia Avenue. Since he will visit all his relatives very often, he wants to find a house close to them.

Indeed, Vito wants to minimize the total distance to all of his relatives and has blackmailed you to write a program that solves his problem.

Input

The input consists of several test cases. The first line contains the number of test cases.

For each test case you will be given the integer number of relatives r ($0 < r < 500$) and the street numbers (also integers) $s_1, s_2, \ldots, s_i, \ldots, s_r$ where they live ($0 < s_i < 30,000$). Note that several relatives might live at the same street number.

Output

For each test case, your program must write the minimal sum of distances from the optimal Vito's house to each one of his relatives. The distance between two street numbers s_i and s_j is $d_{ij} = |s_i - s_j|$.

Sample Input

```
2
2 2 4
3 2 4 6
```

Sample Output

```
2
4
```

4.6.2 Stacks of Flapjacks

PC/UVa IDs: 110402/120, **Popularity:** B, **Success rate:** high **Level:** 2

Cooking the perfect stack of pancakes on a grill is a tricky business, because no matter how hard you try all pancakes in any stack have different diameters. For neatness's sake, however, you can sort the stack by size such that each pancake is smaller than all the pancakes below it. The size of a pancake is given by its diameter.

Sorting a stack is done by a sequence of pancake "flips." A flip consists of inserting a spatula between two pancakes in a stack and flipping (reversing) *all* the pancakes on the spatula (reversing the sub-stack). A flip is specified by giving the position of the pancake on the bottom of the sub-stack to be flipped relative to the entire stack. The bottom pancake has position 1, while the top pancake on a stack of n pancakes has position n.

A stack is specified by giving the diameter of each pancake in the stack in the order in which the pancakes appear. For example, consider the three stacks of pancakes below in which pancake 8 is the top-most pancake of the left stack:

8	7	2
4	6	5
6	4	8
7	8	4
5	5	6
2	2	7

The stack on the left can be transformed to the stack in the middle via *flip(3)*. The middle stack can be transformed into the right stack via the command *flip(1)*.

Input

The input consists of a sequence of stacks of pancakes. Each stack will consist of between 1 and 30 pancakes and each pancake will have an integer diameter between 1 and 100. The input is terminated by end-of-file. Each stack is given as a single line of input with the top pancake on a stack appearing first on a line, the bottom pancake appearing last, and all pancakes separated by a space.

Output

For each stack of pancakes, your program should echo the original stack on one line, followed by a sequence of flips that results in sorting the stack of pancakes so that the largest pancake is on the bottom and the smallest on top. The sequence of flips for each stack should be terminated by a 0, indicating no more flips necessary. Once a stack is sorted, no more flips should be made.

Sample Input

```
1 2 3 4 5
5 4 3 2 1
5 1 2 3 4
```

Sample Output

```
1 2 3 4 5
0
5 4 3 2 1
1 0
5 1 2 3 4
1 2 0
```

4.6.3 Bridge

PC/UVa IDs: 110403/10037, **Popularity:** B, **Success rate:** low **Level:** 3

A group of n people wish to cross a bridge at night. At most two people may cross at any time, and each group must have a flashlight. Only one flashlight is available among the n people, so some sort of shuttle arrangement must be arranged in order to return the flashlight so that more people may cross.

Each person has a different crossing speed; the speed of a group is determined by the speed of the slower member. Your job is to determine a strategy that gets all n people across the bridge in the minimum time.

Input

The input begins with a single positive integer on a line by itself indicating the number of test cases, followed by a blank line. There is also a blank line between each two consecutive inputs.

The first line of each case contains n, followed by n lines giving the crossing times for each of the people. There are not more than 1,000 people and nobody takes more than 100 seconds to cross the bridge.

Output

For each test case, the first line of output must report the total number of seconds required for all n people to cross the bridge. Subsequent lines give a strategy for achieving this time. Each line contains either one or two integers, indicating which person or people form the next group to cross. Each person is indicated by the crossing time specified in the input. Although many people may have the same crossing time, this ambiguity is of no consequence.

Note that the crossings alternate directions, as it is necessary to return the flashlight so that more may cross. If more than one strategy yields the minimal time, any one will do.

The output of two consecutive cases must be separated by a blank line.

Sample Input

```
1

4
1
2
5
10
```

Sample Output

```
17
1 2
1
5 10
2
1 2
```

4.6.4 Longest Nap

PC/UVa IDs: 110404/10191, **Popularity:** B, **Success rate:** average **Level:** 1

Professors lead very busy lives with full schedules of work and appointments. Professor P likes to nap during the day, but his schedule is so busy that he doesn't have many chances to do so.

He *really* wants to take one nap every day, however. Naturally, he wants to take the longest nap possible given his schedule. Write a program to help him with the task.

Input

The input consists of an arbitrary number of test cases, where each test case represents one day.

The first line of each case contains a positive integer $s \leq 100$, representing the number of scheduled appointments for that day. The next s lines contain the appointments in the format *time1 time2 appointment*, where *time1* represents the time which the appointment starts and *time2* the time it ends. All times will be in the `hh:mm` format; the ending time will always be strictly after the starting time, and separated by a single space.

All times will be greater than or equal to 10:00 and less than or equal to 18:00. Thus your response must be in this interval as well; i.e., no nap can start before 10:00 and last after 18:00.

The appointment can be any sequence of characters, but will always be on the same line. You can assume that no line is be longer than 255 characters, that $10 \leq hh \leq 18$ and that $0 \leq mm < 60$. You *cannot* assume, however, that the input will be in any specific order, and must read the input until you reach the end of file.

Output

For each test case, you must print the following line:

`Day #`d`: the longest nap starts at` $hh : mm$ `and will last for [`H `hours and]` M `minutes.`

where d stands for the number of the test case (starting from 1) and $hh : mm$ is the time when the nap can start. To display the length of the nap, follow these rules:

1. If the total time X is less than 60 minutes, just print "X `minutes`."
2. If the total duration X is at least 60 minutes, print "H `hours and` M `minutes`," where

$$H = X \div 60 \quad \text{(integer division, of course) and} \quad M = X \bmod 60.$$

You don't have to worry about correct pluralization; i.e., you must print "`1 minutes`" or "`1 hours`" if that is the case.

The duration of the nap is calculated by the difference between the ending time and the beginning time. That is, if an appointment ends at 14:00 and the next one starts at 14:47, then you have 14:47 – 14:00 = 47 minutes of possible napping.

If there is more than one longest nap with the same duration, print the earliest one. You can assume the professor won't be busy all day, so there is always time for at least one possible nap.

Sample Input

```
4
10:00 12:00 Lectures
12:00 13:00 Lunch, like always.
13:00 15:00 Boring lectures...
15:30 17:45 Reading
4
10:00 12:00 Lectures
12:00 13:00 Lunch, just lunch.
13:00 15:00 Lectures, lectures... oh, no!
16:45 17:45 Reading (to be or not to be?)
4
10:00 12:00 Lectures, as everyday.
12:00 13:00 Lunch, again!!!
13:00 15:00 Lectures, more lectures!
15:30 17:15 Reading (I love reading, but should I schedule it?)
1
12:00 13:00 I love lunch! Have you ever noticed it? :)
```

Sample Output

```
Day #1: the longest nap starts at 15:00 and will last for 30 minutes.
Day #2: the longest nap starts at 15:00 and will last for 1 hours and 45 minutes.
Day #3: the longest nap starts at 17:15 and will last for 45 minutes.
Day #4: the longest nap starts at 13:00 and will last for 5 hours and 0 minutes.
```

4.6.5 Shoemaker's Problem

PC/UVa IDs: 110405/10026, **Popularity:** C, **Success rate:** average **Level:** 2

A shoemaker has N orders from customers which he must satisfy. The shoemaker can work on only one job in each day, and jobs usually take several days. For the ith job, the integer T_i ($1 \leq T_i \leq 1,000$) denotes the number of days it takes the shoemaker to finish the job.

But popularity has its price. For each day of delay before starting to work on the ith job, the shoemaker has agreed to pay a fine of S_i ($1 \leq S_i \leq 10,000$) cents per day. Help the shoemaker by writing a program to find the sequence of jobs with minimum total fine.

Input

The input begins with a single positive integer on a line by itself indicating the number of the test cases, followed by a blank line. There is also a blank line between two consecutive cases.

The first line of each case contains an integer reporting the number of jobs N, where $1 \leq N \leq 1,000$. The ith subsequent line contains the completion time T_i and daily penalty S_i for the ith job.

Output

For each test case, your program should print the sequence of jobs with minimal fine. Each job should be represented by its position in the input. All integers should be placed on only one output line and each pair separated by one space. If multiple solutions are possible, print the first one in lexicographic order.

The output of two consecutive cases must be separated by a blank line.

Sample Input

```
1

4
3 4
1 1000
2 2
5 5
```

Sample Output

```
2 1 3 4
```

4.6.6 CDVII

PC/UVa IDs: 110406/10138, **Popularity:** C, **Success rate:** low **Level:** 2

Roman roads are famous for their sound engineering. Unfortunately, sound engineering does not come cheap, and some modern neo-Caesars have decided to recover the costs through automated tolling.

A particular toll highway, the CDVII, has a fare structure that works as follows: travel on the road costs a certain amount per km traveled, depending on the time of day when the travel begins. Cameras at every entrance and every exit capture the license numbers of all cars entering and leaving. Every calendar month, a bill is sent to the registered owner for each km traveled (at a rate determined by the time of day), plus one dollar per trip, plus a two dollar account charge. Your job is to prepare the bill for one month, given a set of license plate photos.

Input

The input begins with an integer on a line by itself indicating the number of test cases, followed by a blank line. There will also be a blank line between each two consecutive test cases.

Each test case has two parts: the fare structure and the license photos. The fare structure consists of a line with 24 non-negative integers denoting the toll (cents/km) from 00:00 to 00:59, the toll from 01:00 to 01:59, and so on for each hour in the day. Each photo record consists of the license number of the vehicle (up to 20 alphanumeric characters), the time and date (`mm:dd:hh:mm`), the word `enter` or `exit`, and the location of the entrance or exit (in km from one end of the highway). All dates will be within a single month. Each "enter" record is paired with the chronologically next record for the same vehicle, provided it is an "exit" record. Unpaired "enter" and "exit" records are ignored. You may assume that no two records for the same vehicle have the same time. Times are recorded using a 24-hour clock. There are not more than 1,000 photo records.

Output

For each test case, print a line for each vehicle indicating the license number and the total bill in alphabetical order by license number. The output of two consecutive cases must be separated by a blank line.

Sample Input

```
1

10 10 10 10 10 10 20 20 20 15 15 15 15 15 15 15 20 30 20 15 15 10 10 10
ABCD123 01:01:06:01 enter 17
765DEF 01:01:07:00 exit 95
```

```
ABCD123 01:01:08:03 exit 95
765DEF 01:01:05:59 enter 17
```

Sample Output

```
765DEF $10.80
ABCD123 $18.60
```

4.6.7 ShellSort

PC/UVa IDs: 110407/10152, **Popularity:** B, **Success rate:** average **Level:** 2

King Yertle wishes to rearrange his turtle throne to place his highest-ranking nobles and closest advisors nearer to the top. A single operation is available to change the order of the turtles in the stack: a turtle can crawl out of its position in the stack and climb up over the other turtles to sit on the top.

Given an original ordering of a turtle stack and a required ordering for the same turtle stack, your job is to find a minimal sequence of operations that rearranges the original stack into the required stack.

Input

The first line of the input consists of a single integer K giving the number of test cases. Each test case consists of an integer n giving the number of turtles in the stack. The next n lines describe the original ordering of the turtle stack. Each of the lines contains the name of a turtle, starting with the turtle on the top of the stack and working down to the turtle at the bottom of the stack. Turtles have unique names, each of which is a string of no more than eighty characters drawn from a character set consisting of the alphanumeric characters, the space character and the period ("."). The next n lines in the input give the desired ordering of the stack, once again by naming turtles from top to bottom. Each test case consists of exactly $2n+1$ lines in total. The number of turtles (n) will be less than or equal to 200.

Output

For each test case, the output consists of a sequence of turtle names, one per line, indicating the order in which turtles are to leave their positions in the stack and crawl to the top. This sequence of operations should transform the original stack into the required stack and should be as short as possible. If more than one solution of shortest length is possible, any of the solutions may be reported.

Print a blank line after each test case.

Sample Input

```
2
3
Yertle
Duke of Earl
Sir Lancelot
Duke of Earl
Yertle
Sir Lancelot
9
```

```
Yertle
Duke of Earl
Sir Lancelot
Elizabeth Windsor
Michael Eisner
Richard M. Nixon
Mr. Rogers
Ford Perfect
Mack
Yertle
Richard M. Nixon
Sir Lancelot
Duke of Earl
Elizabeth Windsor
Michael Eisner
Mr. Rogers
Ford Perfect
Mack
```

Sample Output

```
Duke of Earl

Sir Lancelot
Richard M. Nixon
Yertle
```

4.6.8 Football (aka Soccer)

PC/UVa IDs: 110408/10194, **Popularity:** B, **Success rate:** average **Level:** 1

Football is the most popular sport in the world, even through Americans insist on calling it "*soccer*." A country such as five-time World Cup-winning Brazil has so many national and regional tournaments that is it very difficult to keep track. Your task is to write a program that receives the tournament name, team names, and games played and outputs the tournament standings so far.

A team wins a game if it scores more goals than its opponent, and loses if it scores fewer goals. Both teams tie if they score the same number of goals. A team earns 3 points for each win, 1 point for each tie, and 0 points for each loss.

Teams are ranked according to these rules (in this order):

1. Most points earned.
2. Most wins.
3. Most goal difference (i.e., goals scored – goals against)
4. Most goals scored.
5. Fewest games played.
6. Case-insensitive lexicographic order.

Input

The first line of input will be an integer N in a line alone ($0 < N < 1,000$). Then follow N tournament descriptions, each beginning with a tournament name. These names can be any combination of at most 100 letters, digits, spaces, etc., on a single line. The next line will contain a number T ($1 < T \leq 30$), which stands for the number of teams participating on this tournament. Then follow T lines, each containing one team name. Team names consist of at most 30 characters, and may contain any character with ASCII code greater than or equal to 32 (space), except for "#" and "@" characters.

Following the team names, there will be a non-negative integer G on a single line which stands for the number of games already played on this tournament. G will be no greater than 1,000. G lines then follow with the results of games played in the format:

team_name_1#goals1@goals2#team_name_2

For instance, *Team A*#3@1#*Team B* means that in a game between *Team A* and *Team B*, *Team A* scored 3 goals and *Team B* scored 1. All goals will be non-negative integers less than 20. You may assume that all team names mentioned in game results will have appeared in the team names section, and that no team will play against itself.

Output

For each tournament, you must output the tournament name in a single line. In the next T lines you must output the standings, according to the rules above. Should

lexicographic order be needed as a tie-breaker, it must be done in a case-insensitive manner. The output format for each line is shown below:

[*a*]) *Team_name* [*b*]p, [*c*]g ([*d*]-[*e*]-[*f*]), [*g*]gd ([*h*]-[*i*])

where [*a*] is team rank, [*b*] is the total points earned, [*c*] is the number of games played, [*d*] is wins, [*e*] is ties, [*f*] is losses, [*g*] is goal difference, [*h*] is goals scored, and [*i*] is goals against.

There must be a single blank space between fields and a single blank line between output sets. See the sample output for examples.

Sample Input

```
2
World Cup 1998 - Group A
4
Brazil
Norway
Morocco
Scotland
6
Brazil#2@1#Scotland
Norway#2@2#Morocco
Scotland#1@1#Norway
Brazil#3@0#Morocco
Morocco#3@0#Scotland
Brazil#1@2#Norway
Some strange tournament
5
Team A
Team B
Team C
Team D
Team E
5
Team A#1@1#Team B
Team A#2@2#Team C
Team A#0@0#Team D
Team E#2@1#Team C
Team E#1@2#Team D
```

Sample Output

```
World Cup 1998 - Group A
1) Brazil 6p, 3g (2-0-1), 3gd (6-3)
2) Norway 5p, 3g (1-2-0), 1gd (5-4)
3) Morocco 4p, 3g (1-1-1), 0gd (5-5)
4) Scotland 1p, 3g (0-1-2), -4gd (2-6)

Some strange tournament
1) Team D 4p, 2g (1-1-0), 1gd (2-1)
2) Team E 3p, 2g (1-0-1), 0gd (3-3)
3) Team A 3p, 3g (0-3-0), 0gd (3-3)
4) Team B 1p, 1g (0-1-0), 0gd (1-1)
5) Team C 1p, 2g (0-1-1), -1gd (3-4)
```

4.7 Hints

4.1 What is the right version of average to solve Vito's problem: mean, median, or something else?

4.3 Does sorting the people by speed help determine who should be paired up?

4.4 How does sorting help?

4.5 Does it help to sort jobs by their length, or by their penalty, or both?

4.6 Can we convert the date/time information to a single integer to make it easier to manipulate?

4.7 Under what conditions will we *not* have to move a turtle?

4.8 How can we simplify our task of writing a comparison function for such a complicated ranking system?

4.8 Notes

4.2 The problem of sorting pancakes using the minimum number of reversals is notorious as the source of the only research paper Bill Gates ever published [GP79]! Beyond the mathematical interest in the problem, it has an interesting application to reconstructing the evolutionary history between species such as human and mouse. A genome reversal event reverses a run of genes in an organism's DNA. These rare events can cause significant evolutionary impact over long periods of time, so reconstructing the order of the reversals becomes an important problem. See [Ber01, Gus97] for more on genome reversals.